

## [L0-L1 核心本质与设计逻辑]

**L0 一句话本质：**在单文件物理隔离的界限下，通过声明式 SQL 编译为 VDBE 寄存器字节码流，借由 B-Tree 页与 Pager 缓存抽象 I/O，并利用 Journal 或 WAL 模式下的 VFS OS 锁定机制提供跨进程 ACID 强一致性的轻量级嵌入式关系数据库。

## L1 四句话逻辑：

- 词法解析与代码生成 (M1)** 采用定制 Lemon 生成器规避传统 Yacc 重入锁，将 AST 结构优化扁平化并一键生成 VDBE 机器指令。
- VDBE 寄存器解释机 (M2)** 维护游标与伪汇编操作码 (Opcode)，逐行读取 B-Tree 节点以规避传统 Volcano 模型的函数调用堆栈开销。
- B-Tree 与 Pager 缓存控制 (M3)** 实现自适应溢出页对齐与 PCache 缓存锁定，物理封装异构块大小，向虚拟机提供平坦的键值查询。
- 原子回滚与 WAL 共享内存 (M4 & M5)** 配合 VFS OS 文件锁保护数据安全，实现从独占写锁向 WAL 读写并发及 Checkpoint 的物理演进。

## [L2 存储引擎数据流演进拓扑]

• [SQL 字符串] → [Lemon 语法规则 AST] → [Prepared VDBE 字节码缓存] → [VDBE 解释器 Loop] → [VFS 文件读取锁申请] → [PCache 缓冲页映射] → [B-Tree Page 槽位定位] → [WAL 追加记录] → [Checkpoint 定期同步回源数据库]

## [M1.1] Tokenizer 与 Lemon 语法规则解析器 (Lemon Parser Generator)

- 语法规则生成器：**SQLite 不使用 Bison/Yacc，而是使用定制的 Lemon 生成器。Lemon 具有 LALR(1) 语法特性，能生成线程安全且无全局变量的解析器。
- 极小堆栈消耗：**Lemon 语法文件 parse.y 将语法动作参数化，避免了 Yacc 堆栈溢出风险，能有效运行在极低内存 footprints 的嵌入式设备中。

## [M1.2] 代码生成器与查询重写优化 (Code Generator &amp; Query Flattening)

- 指令编译流水：**代码生成器遍历 AST，自顶向下输出 VDBE 操作码字节码。
- 查询扁平化：**优化阶段会对多表 Join 和子查询进行查询打平 (Query Flattening)，将内嵌的 SELECT 语句合并到外层，消除 VDBE 虚拟机在执行时的临时表分配开销。

## [M1.3] 索引选择与代价评估模型 (SQLite Cost-Based Index Selector)

- 启发时代价评估：**查询规划器评估全表扫描与索引扫描的相对代价。
- 表达式索引支持：**支持对计算表达式建立索引 (Expression Indexes)，在 CBO 阶段直接替换 AST 节点，避免 VDBE 运行时动态计算。

## [M1.4] Prepared Statement 缓存机制 (Prepared Statement Cache)

- 预编译重用：**通过 sqlite3\_prepare\_v2 将 SQL 编译为 VDBE 伪代码程序，并将参数占位符 (如 ?) 留空。
- 零分配绑定：**运行时使用 sqlite3\_bind\_\* 填充参数，用 sqlite3\_step 循环执行。规避了重复的分词、语法分析及计划生成开销。

## [M2.1] VDBE 核心架构与操作码循环 (VDBE Architecture &amp; Opcode Loop)

- 软 CPU 核心：**VDBE 是一个基于寄存器的软虚拟机。它将 SQL 逻辑扁平化为字节码指令。
- 分发解释循环：**sqlite3VdbeExec() 内部是一个巨型的 switch-case 语句循环，通过计数器 PC 逐条提取并执行操作码，避免了 C++ 算子虚函数调用。

## [M2.2] VDBE 寄存器类型与动态存储 (VDBE Registers &amp; Dynamic Typing)

- 动态值类型：**VDBE 的每个寄存器由 Mem 结构体表示。支持 NULL、INTEGER、REAL、TEXT、BLOB 动态类型，并可携带额外指针。
- 弱类型转换：**在执行加法或比较时，VDBE 会根据列的亲性和 (Affinity) 在运行时自动将文本数字转换为整型或浮点数，保证了无 Schema 限制的灵活性。

## [M2.3] 游标寻址与 B-Tree 交互操作 (Cursor Operations)

- 句柄物理封装：**VDBE 使用游标 (Cursor) 访问底层的 B-Tree 页。
- B-Tree 指针寻址：**指令 OpenRead 分配游标指向特定的表或索引，SeekRowid 在 B-Tree 中二分查找主键，Column 从游标所指的物理 Cell 中提取特定列值，规避了行重建。

## [M2.4] 虚拟机子程序与协程调度 (Bytecode Subroutines &amp; Yield)

- 上下文流式切换：**VDBE 字节码中没有标准的函数调用，而是使用 Yield 指令作为协程调度。
- 临时表规避：**在处理排序、分组 (GROUP BY) 或复合查询时，VDBE 通过 Yield 唤醒发声端，按需流式消费行数据，最大化压低了内存缓冲占用。

## [M2.5] VDBE 字节码诊断与计划分析 (Explain &amp; Explain Query Plan)

- 控制台计划诊断：**开发者输入 EXPLAIN QUERY PLAN <SQL> 可以诊断索引命中路径 (如 USING INDEX 或 SCAN TABLE)。
- 字节码反汇编：**使用 EXPLAIN <SQL> 则可以直接输出 VDBE 内部完整的字节码，如 Init、OpenRead、Rewind、Column、ResultRow 等。

## [M3.1] B-Tree 物理页面布局 (B-Tree Page Layout)

- 异构节点对齐：**B-Tree 页面大小可配置为 512B 到 64KB。页头部包含 8 到 12 字节元数据，指示当前页是叶子还是内部节点。
- 单元物理存储：**每条记录打包为 Cell (包含 Key 和 Payload)，Cell 的物理偏移保存在页头部的 Cell 指针数组中，数组向右增长，而 Cell 实体则从页面尾部向左增长。

## [M3.2] 溢出页链表组织 (Overflow Pages)

- 小页面保护机制：**如果单个 Cell 的 Payload 大小超过当前页面的限定阈值 (由页大小计算得出)，SQLite 会将超限的数据抽取并存储到独立的 Overflow 溢出页中。
- 链表指针维护：**叶子节点 Cell 仅保留部分头部数据和指向首个溢出页的 4 字节页号指针，从而维持了 B-Tree 索引页面极高的检索命中率。

## [M3.3] Pager 逻辑层与页面缓存管理 (Pager Layer &amp; PCache)

- 物理页缓冲映射：**Pager 模块介于 VDBE/B-Tree 和 OS 之间。它将物理文件的磁盘块抽象为内存中的 Page 对象，并由 PCache 结构管理。
- Pin 与脏标记：**Pager 跟踪每个页面的状态：Clean (未修改)、Dirty (需写日志)、Pinned (正在被 VDBE 使用，不允许被淘汰)。

## [M3.4] Free-list 空闲页链表与 VACUUM 整理 (Free-list &amp; Compaction)

- 重用不收缩：**当删除数据释放页面时，SQLite 并不立即截断物理文件，而是将页号挂载到全局的 Free-list (空闲页链表) 中用于后续重用。
- 物理收缩与碎片重整：**通过 VACUUM 指令在本地创建一个零碎片的临时数据库文件，将所有活跃页拷贝进去并替换原文件，实现物理大小的真正收缩。

## [意] SQLite 并发模式与日志机制折衷与异常防范矩阵 (SQLite Concurrency &amp; Journaling Trade-offs)

SQLite 可以在多进程间共享同一个连接句柄以提高并发效率：多进程共享同一个 sqlite3 数据库句柄或底层的 VFS 文件描述符会导致文件状态紊乱，极易引发数据库损坏或进程死锁崩溃。 → 每个进程必须使用独立的连接句柄。在多线程下，使用 sqlite3\_open\_v2 并开启共享缓存模式，或每个线程持有一个独立连接以保证安全性。

直接删除 -journal 或 -wal 文件可以快速修复被锁定的数据库：直接删除活跃事务中的 -journal 或 -wal 临时日志文件会导致 Pager 引擎丢失原页快照，导致主数据库文件永久陷入半写入损坏状态。 → 禁止手动干预日志文件。若发生数据库锁定，应排查持有锁的挂起进程并将其终止，让 SQLite 引擎自动读取日志文件执行崩溃恢复。

将 synchronous 设置为 OFF 可以无痛提升百倍写入吞吐：同步配置为 OFF 会完全绕过操作系统文件的 fsync 物理刷盘指令，此时若发生主机宕机或断电，物理文件头将发生彻底损坏且无法恢复。 → 开启 WAL 模式并将 synchronous 配置为 NORMAL。此时写操作只在 WAL 追加时不强制 fsync，仅在 Checkpoint 脏页回源时 fsync，兼顾吞吐与持久性。

## [M4.1] -journal 回滚日志物理结构 (Rollback Journal Layout)

- **原始快照留存**: 回滚模式下，修改任何数据库页之前，Pager 会把该页的原始明文数据拷贝写入物理后缀为 `~journal` 的文件中。
- **日志头校验**: Journal 文件头部包含 8 字节幻数、当前日志的记录页数，以及原始数据库的物理大小，用于在崩溃恢复时进行主数据库尺寸裁剪对齐。

## [M4.2] 回滚模式事务提交与崩溃恢复 (Rollback Mode Commit &amp; Recovery)

- **双阶段物理刷盘**:
  1. **写日志**: 将修改前页写入 Journal，并对 Journal 文件执行 `fsync` 刷盘。
  2. **写主库**: 修改内存页并回写至 `.db` 主文件，对主文件执行 `fsync`。最后删除或截断 Journal 文件。
- **崩坏自动回滚**: 如果中途发生断电，重启后 Pager 发现 `~journal` 文件存在且完好，会读取其内容覆盖回主文件，还原来完成修改。

## [M4.3] 回滚日志状态的锁变迁 (Lock Escalation in Rollback Mode)

- **轻量级升级路径**:
  1. **SHARED**: 可读但不可写。
  2. **RESERVED**: 准备写，允许其他连接继续读取，但禁止其他 **RESERVED** 锁。
  3. **PENDING**: 准备提交写，阻止新的 **SHARED** 读锁，等待已有的读者退出。
  4. **EXCLUSIVE**: 独占文件锁，允许写入主文件。

## [M4.4] 独占写冲突与多进程锁死 (Exclusive Write Concurrency Limit)

- **单写瓶颈**: 由于 **EXCLUSIVE** 锁是物理文件锁，一旦某一进程进入 **EXCLUSIVE** 状态开始修改主文件，其他所有进程的 `sqlite3_step` 读操作都将被文件系统物理阻塞，抛出 `SQLITE_BUSY`。

## [M5.1] -wal 日志文件物理结构 (WAL Log Layout)

- **日志增量追加**: WAL 模式下，修改过的页面不再写入原数据库文件，而是直接追加物理写入 `~wal` 文件中。
- **帧首部对齐**: WAL 文件以 32 字节的 WAL Header 开头，随后是多个 WAL Frame。每个 Frame 包含 24 字节 Frame Header (包含 PageID 和 CRC 校验) 以及物理页的数据主体。

## [M5.2] -shm 共享内存索引文件 (WAL Shared Memory Index)

- **哈希加速定位**: 为规避读事务在几十字节的 `~wal` 日志中顺序搜索最新页面的开销，SQLite 在同目录下生成 `-shm` 共享内存文件。
- **多进程虚拟共享**: `-shm` 将 WAL 中的 Frame 序号与 Page ID 组织成内存哈希索引，不同进程通过 `mmap` 直接读取，实现微秒级检索重定向。

## [M5.3] WAL 读写并发非阻塞控制 (WAL Read/Write Concurrency)

- **快照隔离现实**: 写事务向 `~wal` 文件尾部追加帧，不修改 `.db` 文件。
- **双向寻址读**: 读事务启动时，记录当前 WAL 的最大帧号。读取页面时，先在 `-shm` 索引中二分查找，若在快照范围内有该页，则从 `~wal` 读取，否则读取主 `.db`，实现了读写并发共存。

## [M5.4] Checkpoint 定期脏页回源机制 (Checkpointing)

- **脏页回源对齐**: Checkpoint 操作由 Pager 的后台或调用线程触发，负责将 `~wal` 文件中的最新页面帧顺序拷贝回源数据库 `.db` 文件中。
- **重写写入点**: 同步完成后，重置 WAL 的头部指针。如果有读事务仍在引用旧的 WAL 帧，则只能执行 **PASSIVE** 增量同步，直到旧读者退出才能安全重写。

## [M5.5] WAL 物理写屏障与 fsync 规则 (WAL fsync Rules)

- **双重持久屏障**:
  1. **事务提交**: 向 `~wal` 追写完 Commit 帧后，仅对 `~wal` 文件执行 `fsync`，即宣告提交成功。
  2. **源库合并**: 在 Checkpoint 期间将页回写主库后，先对 `.db` 主文件执行 `fsync`，然后再向 WAL 写入标记，最后更新 `-shm`。

## [M5.6] 追尾避免与 WAL 重写重置 (WAL Wraparound)

- **安全覆盖机制**: 当 WAL 文件不断变大，SQLite 会检查 `-shm` 中所有活跃读连接的最小读取帧 LSN。
- **零开销重置**: 如果该最小帧号已经覆盖了当前的 WAL 头部，写事务可以直接从 WAL 文件偏移 0 处重新写入，避免了频繁的文件物理删除与重新分配。

## [M6.1] VFS 虚拟文件系统抽象层 (VFS Abstraction Layer)

- **多平台物理抽象**: VFS (虚拟文件系统) 将操作系统的文件 I/O 隔离。
- **接口映射子集**: `sqlite3_vfs` 结构体定义了 `xOpen`, `xDelete`, `xRead`, `xWrite`, `xLock`, `xUnlock` 等虚函数。不同平台 (如 `unix` 或 `win32`) 通过实现这些函数，保证了内核逻辑 100% 平台无关。

## [M6.2] Windows 与 Unix 锁定原语物理映射 (Win32 vs POSIX File Locking)

- **Unix 字节范围锁**: Unix VFS 利用 `fcntl` 字节范围锁，通过在数据库文件的特定偏移区间 (如从 1073741824 字节开始的一段空间) 申请排他锁来模拟 **SHARED/PENDING/EXCLUSIVE** 状态。
- **Win32 强制锁**: Windows 平台则使用 `LockFileEx` 来锁定相同的虚拟字节区间，保障跨进程并发安全。

## [M6.3] 多线程与多进程锁隔离屏障 (Thread vs Process Locks)

- **互斥体屏障**: 为了保护库内全局状态，在多线程环境下使用互斥锁 (Mutex)。
- **文件锁屏障**: 在跨进程 (如不同的 Python 或 PHP 脚本) 并发访问同一个单文件时，使用 VFS 文件锁协调事务隔离，保证双重防护无死角。

## [M6.4] mmap 内存映射物理 I/O (mmap I/O Optimization)

- **零拷贝加速**: 通过配置 `PRAGMA mmap_size`，SQLite 允许 VFS 将物理数据库文件直接通过 `mmap` 映射到宿主进程的虚拟地址空间。
- **旁路 Buffer 拷贝**: 只读操作可以直接在指针上读取字节，绕过了操作系统的 `Read` 系统调用和内核缓冲与用户空间的二次拷贝，大幅节约 CPU 周期。

## [M6.5] 加密扩展与物理页安全性加密 (SQLite Encryption Extension SEE)

- **页级块加密**: 由于 SQLite 按页存储，SEE 和 SQLCipher 在 Pager 写入磁盘前，对整页 (除第一页的前 100 字节文件头外) 进行 AES 加密。
- **读页解密闭环**: 物理页读入 Pager 缓存后，在内存中完成解密。外部 VDBE 看到的是明文，而磁盘上则是高强度密文，保护了本地单文件安全。

## [貳] Zone T SQLite 性能调优与诊断实验室 (SQLite Diagnostic &amp; Tuning Reference Laboratory)

## T1 SQLite 典型参数调优对照表

```
--- (---)
:--
PRAGMA journal_mode = WAL; (WAL)
切换为预写日志模式。读写完全不互斥，是解决多进程并发读取及高频写入的首选模式。
PRAGMA synchronous = NORMAL; (NORMAL (2))
在 WAL 模式下配置为 NORMAL，事务提交时仅 fsync WAL 日志，极大缓解了机械盘与低端 SSD 的刷盘卡顿。
PRAGMA cache_size = ~2000; (~2000 (约 2MB 缓存))
负值表示以 KB 为单位分配 Pager 页缓存大小，正值表示缓存的页数。根据进程内存余量可调大至 ~10000 (10MB)。
PRAGMA temp_store = MEMORY; (MEMORY (2))
将排序和临时表 (Temp Tables) 强制存放在内存中而非磁盘上，大幅消除 VDBE 处理大排序时的随机 I/O。
```

## T2 经典性能监控与诊断 PRAGMA 指令字典

```
• 1. 获取物理计划与字节码诊断
sql
-- 反汇编并输出 VDBE 字节码伪指令，用于深度剖析 VM 寄存器状态与游标跳转
EXPLAIN SELECT name, age FROM users WHERE age > 18;
-- 查看高维度的物理执行计划，评估是否命中索引以及多表 Join 排序
EXPLAIN QUERY PLAN SELECT * FROM orders JOIN users ON orders.uid = users.id;

• 2. 全库健康完整性校验
sql
-- 深度扫描 B-Tree 页面、Cell 偏移和空闲页，验证物理结构是否损坏
PRAGMA integrity_check;
```

资料来源: SQLite Internals 官方规范与系统实现。排版采用流式 extarticle + multicols 多栏高密度格式，无第三方水印，商品级分发纯净度。